Entwicklungsbericht Mini-Challenge

KI-gestützte Entwicklung einer Fallschirmsprung-Tracking-App

Damian Szedalik

5. Dezember 2025

Inhaltsverzeichnis

1 Einleitung **3**
1.1 Projektbeschreibung . . . 3
1.2 Überblick über die entwickelte Lösung . . . 3
1.3 Ergebnis im Vergleich zu den Zielen . . . 5

2 Anforderungsszenario und dessen Entwicklung **6**
2.1 Initiales Anforderungsszenario . . . 6
2.2 Entwicklung des Szenarios während des Projekts . . . 6
2.3 Finale Einordnung des Projekts . . . 6
2.4 Reflexion . . . 7

3 Entwicklungsstrategie und KI-Einsatz **8**
3.1 Durchgeführtes Vorgehen mit Beispielen . . . 8
3.2 Was hat funktioniert? . . . 9
3.3 Wo musste das Vorgehen angepasst werden? . . . 9
3.4 Was hat nicht funktioniert? . . . 10
3.5 Umgang mit Modellgrenzen . . . 10

4 Verständnis und Ownership **12**
4.1 Verständnistiefe der verschiedenen Komponenten . . . 12
4.2 Methoden zur Herstellung von Verständnis . . . 12
4.3 Ownership und Verantwortung . . . 13
4.4 Kritische Reflexion . . . 13
4.5 Risiken und Sicherheit . . . 13

5 Qualitätssicherung **14**
5.1 Teststrategien . . . 14
5.2 Manuelle Validierung . . . 14
5.3 Github . . . 14
5.4 Testing durch Personen . . . 14

6 Fazit und Lessons Learned **15**
6.1 Wichtigste Erkenntnisse 15
6.2 Was ich anders machen würde 15
6.3 Perspektive auf KI-gestützte Entwicklung 15
6.4 Empfehlungen für andere Studierende 15

A Anhänge **17**

1 Einleitung

1.1 Projektbeschreibung

Im Rahmen der Mini-Challenge im Modul KIP entwickle ich eine Fallschirmsprung-Tracking-App, die auf Android, iOS und WearOS verfügbar ist. Die Anwendung ermöglicht es Fallschirmspringer:innen, ihre Sprünge zu erfassen, persönliche Profile und Ausrüstung zu verwalten sowie Statistiken über die absolvierten Sprünge einzusehen.

Kernfunktionen der finalen App sind:

- **Sprungerfassung**: Automatische Datum-/Zeitangabe mit Möglichkeit zur nachträglichen Anpassung, manuelle Erfassung von Absprunghöhe, Ort (mit GPS und Kartenauswahl) sowie Sprungtyp und -methode.
- **Wetter-Integration**: Automatischer Abruf von Wetterdaten (Temperatur, Wind-Geschwindigkeit/-Richtung, etc.) via Open-Meteo API inkl. historischer Daten für nachgetragene Sprünge.
- **Freefall-Detektion**: Automatische Erkennung von Exit und Deployment via Beschleunigungssensoren mit Berechnung von Freifall-Dauer und Maximalgeschwindigkeit.
- **Profil und Ausrüstung**: Verwaltung persönlicher Daten (Name, Lizenz) und Speichern des eigenen Equipments (Schirm, Gurtzeug, Reserve etc.).
- **Statistik und Übersicht**: Gesamtzahl der Sprünge, durchschnittliche Höhe, Sprungtypen-Verteilung, Karte mit Sprungorten.
- **Achievements**: Freischaltbare Erfolge basierend auf Sprunganzahl und Meilensteinen.
- **WearOS-Unterstützung**: Vollständig angepasste UI für Galaxy Watch und andere runde Smartwatch-Displays mit eigenständiger Sprungerfassung und Freefall-Detektion.

Technisch wurde die App mit **Flutter/Dart** für das Frontend (Cross-Platform für Android, iOS und WearOS) sowie einem **Python/FastAPI**-Backend mit **SQLite** als Datenbank umgesetzt. Ein zentrales Ziel war das reflektierte Arbeiten mit Cursor.

1.2 Überblick über die entwickelte Lösung

Die finale App besteht aus zwei Hauptkomponenten:

Frontend (Flutter):

- 4 Hauptscreens (Home/Statistik, Sprungerfassung, Equipment, Einstellungen)
- 4 WearOS-spezifische Screens mit adaptiver runder UI
- Riverpod für State Management
- Services für API-Kommunikation, Geocoding, Wetter, Freefall-Detektion
- Unit- und Widget-Tests

Backend (Python/FastAPI):

- RESTful API mit 5 Endpunkt-Gruppen (Jumps, Equipment, Profile, Statistics, Weather)
- SQLite
- Pydantic-Schemas für Request/Response-Validierung
- Integration der Open-Meteo API für Wetterdaten
- Pytest-Tests

In den Abbildungen 8, 9, 10, 11, 12 ist die fertige App zu sehen.

1.3 Ergebnis im Vergleich zu den Zielen

Geplante Funktion	**Priorität**	**Status**
Sprungerfassung mit Datum/Zeit	Definitiv	✓
Ort mit GPS/Kartenauswahl	Definitiv	✓
Manuelle Höhenangabe	Definitiv	✓
Checkliste vor Sprung	Definitiv	→ geändert
Profil und persönliche Daten	Definitiv	✓
Equipment-Verwaltung	Definitiv	✓
Gesamtzahl Sprünge/Statistik	Definitiv	✓
Frontend: React	Definitiv	→ Flutter
Backend: Python	Definitiv	✓
Datenbank: SQLite	Definitiv	✓
Wetter-API Integration	Optional	✓
Karte mit Sprungorten	Optional	✓
Bewertung/Notizen zum Sprung	Optional	✓
WearOS/Galaxy Watch Support	Optional	✓
Warnung bei schlechtem Wetter	Optional	×
Video-Link zum Sprung	Optional	×
Erweiterte Diagramme	Optional	teilw.

Tabelle 1: Übersicht der geplanten vs. umgesetzten Funktionen

Wesentliche Änderungen:

- **Technologie-Wechsel React zu Flutter**: React Native führte zu wiederkehrenden Build-Problemen, die ich nicht in akzeptablem Zeitraum lösen konnte. Mit Flutter konnte ich Android, iOS und WearOS aus einer Codebasis bedienen.
- **Equipment-Checkliste**: Statt einer Checkliste dient der Bereich nun zur Erfassung, welches Equipment bei einem Sprung genutzt wurde.
- **Freefall-Detektion und Achievements**: Beide Features waren anfangs nicht geplant, sind aber im Verlauf hinzugekommen.

Nicht realisiert wurden Wetter-Warnungen, da sie konzeptionell nicht mehr gut in die App-Struktur passten, sowie Video-Links aus Zeitgründen nicht eingebaut.

2 Anforderungsszenario und dessen Entwicklung

2.1 Initiales Anforderungsszenario

Das initiale Anforderungsszenario sah eine App vor, welche:

- auf Android und iOS lauffähig ist (mit React)
- Sprünge mit diversen Daten dazu erfasst
- ein einfaches Profil- und Equipment-Management bietet
- grundlegende Statistiken anzeigt

2.2 Entwicklung des Szenarios während des Projekts

Das Anforderungsszenario hat sich verändert:

Technologie-Wechsel React zu Flutter

Trotz vielen Versuchen waren die React Native Builds nicht stabil. Auf Empfehlung eines Mitstudenten bin ich auf Flutter umgestiegen. Nach einem kurzen Onboarding verlief der Umstieg fast reibungslos. Gleichzeitig erweiterte dies das Szenario auf WearOS (im Nachhinein herausgefunden).

Freefall-Detektion

Im Verlauf erschien es sinnvoll, neben Höhe und Ort auch Freifall-Dauer und Maximalgeschwindigkeit zu erfassen. Da reale Tests nicht möglich waren, habe ich gleichzeitig eine Fall-Simulation eingebaut.

Achievements

Aus der anfänglichen Profilseite ist eine Ansicht mit Lizenzdaten und Achievements entstanden.

2.3 Finale Einordnung des Projekts

Die finale Einordnung ist höher als am Anfang geschätzt:

- **Komplexität**: Hoch
  - Cross-Platform (Android, iOS, WearOS)
  - Sensor-Integration für Freefall-Detektion
  - Externe API-Integration (Open-Meteo)
  - Adaptives UI für verschiedene Geräte
- **Verständnisanspruch**: Hoch
  - Flutter/Dart
  - FastAPI/SQLAlchemy
  - Riverpod State Management

2.4 Reflexion

Die anfängliche Einschätzung war zu optimistisch bezüglich der Komplexität. Insbesondere React erwies sich als Problematisch. Auch nach dem Umstieg auf Flutter in Kombination mit Riverpod gibt es weiterhin viele Fragezeichen in Bezug auf den Code.

Der Fokus auf Kernfunktionen, dann iterative Erweiterung, hat aber stark dabei geholfen zu verstehen wie alles in der App zusammenhängt.

3 Entwicklungsstrategie und KI-Einsatz

Ich habe den Cursor Agent für grössere Programmier-Schritte genutzt, Cursor Chat für Erklärungen zum Code, Cursor-Bot in Github um Rückmeldungen zu meinem Pull-Request zu erhalten und teilweise Code-Completion bei kleineren manuellen Anpassungen.

Grundsätzliche Strategie:

1. Mit Cursor eine grundlegende Projektstruktur erstellen
2. Funktionen einzeln und nacheinander einbauen (nicht alles auf einmal)
3. Regelmässig ausgeben lassen, welche Komponenten hinzugefügt wurden, um Ownership herzustellen
4. Bei Problemen: Debug-Logs einbauen lassen und mit konkretem Output prompten

Zu Beginn waren meine Prompts eher spontan, zum Beispiel:

> Jetzt baue eine Equipment-Funktion ein, wobei man verschiedene Equipments erfassen kann und in der Sprungerfassung auswählen kann.

Gegen Ende eher strukturierter:

> Kontext: Ich entwickle eine Fallschirmsprung-Tracking-App in Flutter mit Riverpod. Es gibt bereits eine Seite zur Sprungerfassung und ein einfaches Profil mit Basisdaten. Bisher gibt es aber noch keine Verwaltung von Equipment.
> Ziel: Implementiere eine Equipment-Verwaltung, bei der ich mehrere Equipmentteile (z.B. Hauptschirm, Reserve, Gurtzeug) anlegen kann und in der Sprungerfassung schlussendlich anwählen kann.
> Beachte: - Bestehende Sprungerfassung muss weiterhin so funktionieren wie bisher. - Daten sollen in der bestehenden SQLite-Datenbank gespeichert werden.
> Ergebnis: - Neue Screens/Widgets für Equipment-Übersicht und -Erfassung. - Anpassung des Jump-Modells, sodass ein Equipment-Set referenziert wird.

3.1 Durchgeführtes Vorgehen mit Beispielen

Beispiel 1: Projektstruktur

Abbildung 1 zeigt den Output vom ersten Prompt, welcher die grundlegende Architektur aufbauen soll. Ich habe bewusst nur die definitiv geplanten Funktionen aufgelistet, um den Scope überschaubar zu halten:

> Ich möchte eine Fallschirmsprung-Tracking-App entwickeln...
>
> 1. Entwirf für die unten gelisteten Funktionen eine sinnvolle Projektstruktur nach Best Practices.
> 2. Richte ein grundlegendes Test-Setup ein.
> 3. Erkläre kurz, warum du diese Struktur gewählt hast.

Cursor hat eine ausführliche Struktur mit FastAPI-Backend und React- Frontend erstellt, inklusive Erklärung der Architekturentscheidungen.

Beispiel 2: Iterative Feature-Entwicklung

Nach dem Umstieg auf Flutter wurden Features einzeln nachgerüstet. Beispiel Wetter-Integration (mittelmässig guter Prompt):

> Bei der Sprungerfassung soll automatisch das Wetter an der angegebenen Location, Zeit und Datum abgerufen und mitabgespeichert werden. Nutze die Open-Meteo API.

Nach einigen Iterationen wegen Unterschieden zwischen Forecast- und Archiv-Wetterdaten funktionierte die Lösung wie in Abbildung 7.

3.2 Was hat funktioniert?

Klare, fokussierte Prompts

Der Umstieg von React auf Flutter hat mit einem einzigen, gut strukturierten Prompt funktioniert:

> Ich habe aktuell ein React Frontend für eine Fallschirmsprung-Tracking-App (Erfassung von Sprüngen, Statistik, Profil, Equipment). Wegen Build-Problemen will ich auf Flutter wechseln.
> - Analysiere das gesamte Projekt - Übernimm alle existierenden Funktionen, welche in React geschrieben sind und schreibe sie in Dart Code - Beachte dabei alle Zusammenhänge zwischen frontend und backend

Debug-Logs

Wenn Cursor sich in Fehler-Loops verfangen hat, war folgender Ansatz erfolgreich:

> Baue Debug-Logs ein, welche helfen herauszufinden woher das Problem stammt. Ich werde dir dann den Output senden.

Dies führte oft zu sofortiger Lösung, da Cursor mit konkretem Output besser arbeiten konnte.

GitHub Cursor-Bot für Code-Reviews

Durch Aktivierung des Cursor-Bots im GitHub-Repository erhielt ich automatisch Kommentare auf Pull Requests (Abbildung 4), die halfen die App zu verbessern.

Wetterintegration

Die Anbindung der Open-Meteo API klappte nach initialem Debugging (historische Daten erfordern andere API-Endpunkte).

3.3 Wo musste das Vorgehen angepasst werden?

Prompt-Grösse:

Anfangs verlangte ich zu viele Änderungen auf einmal. Das führte zu vergessenen Abhängigkeiten, Build-Fehlern und schlechter Nachvollziehbarkeit.

Anpassung: Prompts wurden auf wenige Punkte beschränkt und nach Funktionen gruppiert.

Kontext-Reset bei Loops:

Bei einer fehlerhaften Filterfunktion bin ich mit Cursor im Kreis gelaufen. Die Lösung war ein neuer Chat ohne bisherigen Verlauf, dafür mit klarer Beschreibung von Ist- und Soll-Zustand. Das erste Ergebnis in diesem neuen Chat war korrekt.

Präzisere Problembeschreibung:

Bei einem Location-Eingabe-Bug (Eingabe wurde plötzlich durch eine US-Adresse ersetzt, siehe Abbildung 6) halfen wiederholte vage Beschreibungen nicht weiter. Erst ein Prompt mit genauem Zielverhalten, den ich mir von ChatGPT formulieren liess, führte schnell zur Lösung.

3.4 Was hat nicht funktioniert?

React Native Build:

Das React-Frontend litt unter schwer nachvollziehbaren Build-Fehlern und inkonsistenten Dependencies. Prompts mit Fehlermeldungen führten oft nur zu neuen Fehlern. Ohne tiefe React-Erfahrung und angesichts des Zeitrahmens, habe ich mich dazu entschieden auf Flutter umzusteigen.

Datenbank-Initialisierung:

Daten wurden zwar gespeichert, aber bei App-Neustart nicht geladen. Cursor konnte das Problem nicht lösen. Durch eigenes Nachforschen entdeckte ich einen fehlenden Initialisierungsaufruf im main.py. Vermutlich hatte Cursor im Kontext nur die laufende App im Blick und nicht das Startverhalten.

Freefall-Detektion:

Die Implementierung verlief in mehreren gescheiterten Anläufen:

1. Erster Prompt brach den Build komplett.
2. Nach Korrekturen war die Detektion zwar eingebaut, funktionierte aber nicht.
3. Nebenbei wurden bestehende Datenerfassungen zerstört, Daten nicht gespeichert oder falsch gelesen.

Erst mit gezieltem Einbau von Debug-Logs und schrittweisen Prompts liess sich eine stabile Lösung erzielen. Rückblickend waren die initialen Prompts hier zu ungenau.

WearOS Geolocator-Crash:

Die Geolocator-API ist auf WearOS nicht verfügbar und führte zu Crashes. Die Lösung war, Geolocator auf WearOS komplett zu entfernen und dort nur manuelle Location-Eingabe zu erlauben. Die KI hat dies nicht von sich aus vorgeschlagen.

3.5 Umgang mit Modellgrenzen

Limitierungen erkennen

Hinweise auf Modellgrenzen waren:

- Wiederholte, ähnliche Vorschläge trotz unverändertem Fehler
- Vergessene Abhängigkeiten in anderen Dateien bei änderungen
- Zunehmend inkonsistente Antworten bei sehr langen Chats

Strategien

- Explizite Aufforderung, alle abhängigen Dateien zu aktualisieren
- Regelmässige neue Chats mit komprimiertem Kontext
- Bewusste Entscheidung, manche Fehler (z.B. Datenbank-Init) selber zu lösen

4 Verständnis und Ownership

4.1 Verständnistiefe der verschiedenen Komponenten

Komponente	Verständnis	Begründung
API-Endpunkte (FastAPI)	Mittel	CRUD-Konzept und Routing sind klar, Details von FastAPI-Features nicht vollständig
Pydantic-Schemas	Grösstenteils	Schema-Definition und Validierung sind verständlich
SQLAlchemy-Models	Mittel	Grundprinzip von Modellen und Relationen ist klar
Flutter UI-Screens	Mittel	Widget-Konzept verstanden, komplexe Layout-Optimierungen weniger
Riverpod State Management	Gering	Idee von Providern ist klar, Details der Implementierung aber schwierig
Freefall-Detektion	Grösstenteils	Sensor-Logik und Berechnungen sind nachvollziehbar
Geocoding und Wetter	Mittel	API-Calls und Datenverarbeitung sind verständlich

4.2 Methoden zur Herstellung von Verständnis

Strukturierte Code-Analyse

Nach dem ersten funktionierenden Stand habe ich gepromptet:

> Analysiere den gesamten Code im Projekt.
>
> 1. Liste alle wichtigen Komponenten/Dateien mit kurzer Beschreibung.
> 2. Markiere, was kritisch für die App ist.
> 3. Erstelle eine Prioritätenliste (was ich verstehen muss).

Dies gab mir einen guten Überblick über die Architektur.

Erklärungen bei neuen Features

Bei neuen Feature fügte ich teils hinzu:

> Falls neue Komponenten eingebaut wurden, erkläre mir kurz und klar was sie machen und wie sie eingebaut wurden.

Debugging

Die manuelle Behebung von Fehlern (z.B. Datenbank-Problem) führte zu tieferem Verständnis.

Externe Ressourcen

Für Flutter und Riverpod habe ich Tutorials und Videos verwendet, um ein minimales Grundwissen aufzubauen. Auf tiefes Verständnis von Design-Details der UI und von

Riverpod habe ich verzichtet, da der Aufwand im Rahmen der MC zu hoch gewesen wäre.

4.3 Ownership und Verantwortung

Fühle ich mich als Owner?

Fühle mich unterschiedlich stark verantwortlich:

- **Backend**: Hier fühle ich mich als Owner. Struktur und Datenmodelle kann ich erklären und teilweise leicht anpassen.
- **Frontend-Logik**: Hierzu mittelmässig. Zusammenhänge zwischen Screens, State und API-Aufrufen sind mir nur teilweise klar. Gewisse teile wären durchaus möglich anzupassen, aber nicht weiterentwickeln.
- **Flutter-Widget-Details**: Hier sehe ich mich kaum als Owner, da ich nicht allzuviele Widgets genauer angeschaut habe.

4.4 Kritische Reflexion

Ich hätte gerne ein besseres Verständnis von Riverpod, da dieses die gesamte App zusammenhält. Bei Problemen im State-Management wäre ich ohne KI-Support relativ hilflos. Auch bei den Datenmodellen hätte ein früher vertieftes Verständnis einige Anzeigeprobleme vermutlich verhindert.

Der Trade-off zwischen Geschwindigkeit und Verständnis war für repetitive Flutter-UI und für die initiale Projektstruktur vertretbar. Bei Teilen wie Datenmodellen und State-Management würde ich im Nachhinein mehr Zeit in eigenes Verständnis investieren.

4.5 Risiken und Sicherheit

Risiken aus mangelndem Verständnis:

- Ineffizienter oder schwer wartbarer Dart- und Riverpod-Code
- Fehler in der Freefall-Detektion oder im State-Management könnten zu falschen Statistiken oder unerwartetem Verhalten führen

Cybersecurity-Aspekte:

- Speicherung nur weniger persönlicher Daten (Name, Lizenz)
- Keine API-Keys, da Open-Meteo keine Authentifizierung erfordert
- Bisher nur Lokale SQLite-Datenbank ohne Cloud-Synchronisation

Datenschutz bei KI-Nutzung:

Es werden keine personenbezogene Daten in den Prompts erwähnt.

5 Qualitätssicherung

5.1 Teststrategien

Backend (Python/pytest):

- Tests für alle API-Endpunkte
- Fixtures für Datenbanksetup
- Mocking der externen Weather-API
- Tests für typische Fehler (404, 422, 500)

Frontend (Flutter):

- Unit-Tests für Models (Jump, Weather, FreefallStats)
- Widget-Tests für Hauptscreens
- Mocking der HTTP-Clients

5.2 Manuelle Validierung

- Alle Features auf Android/iOS/wearOS-Emulator getestet
- Edge Cases: Sprünge mit historische Daten, Filter-Kombinationen, Daten löschen, etc. getestet

5.3 Github

- Cursor-Bot-Kommentare als zusätzliche Code-Reviews

5.4 Testing durch Personen

- Ich habe drei Mitstudierende die App testen lassen, um herauszufinden ob die App selbsterklärend/Benutzerfreundlich ist

6 Fazit und Lessons Learned

6.1 Wichtigste Erkenntnisse

Kontext-Management

Wenn man sich mit Cursor im Kreis dreht, hilft ein neuer Chat mit kompaktem, präzisem Prompt und klarer Beschreibung von Ist- und Soll-Zustand. Weniger Verlauf ist oft besser als ein langer, verwirrender Kontext.

Debug-Logs

Cursor kann mit konkretem Output besser arbeiten als mit selbst geschriebener Fehlerbeschreibungen. „Hier ist der Log, was ist falsch?“ funktioniert besser als „Es funktioniert nicht, weil XY.“

Eigenes Verständnis

KI kann Code generieren, aber nicht garantieren, dass er korrekt oder sinnvoll ist. Eigenes Verständnis ist nötig, um zu hinterfragen und zu debuggen.

6.2 Was ich anders machen würde

- Vorher recherchieren, welche Technologie gut ist (React/Flutter)
- Von Anfang an kleinere, klar strukturierte Prompts mit wenigen Punkten schreiben
- Früher testen, statt viele Features am Stück zu implementieren
- Kritische Komponenten wie Datenmodelle und State-Management früher selbst vertieft verstehen (Durch zu späten Verständnisaufbau habe ich viel Zeit verloren mir zuerst den Überblick über den Code zu verschaffen)

6.3 Perspektive auf KI-gestützte Entwicklung

Vorteile

- Enormer Geschwindigkeitsgewinn (Vorallem bei Repetitivem Code und Einarbeitung in neue Frameworks)
- Hilfreich beim Entwerfen von Architektur und beim Refactoring

Grenzen

- Bei komplexen, zusammenhängenden Änderungen weniger zuverlässig
- Kontextgrenzen führen zu inkonsistenten Ergebnissen
- Debugging erfordert weiterhin menschliches Verständnis

6.4 Empfehlungen für andere Studierende

1. Grundlagen zuerst lernen. KI ersetzt eigenes Verständnis nicht.
2. Nicht zu lange an nicht funktionierenden Ansätzen festhalten, sondern bereit sein umzuschwenken.

3. KI-Vorschläge kritisch prüfen und verstehen, statt sie nur zu übernehmen.
4. Kontext regelmässig zurücksetzen, wenn sich der Chat in Schleifen verfängt.

A Anhänge

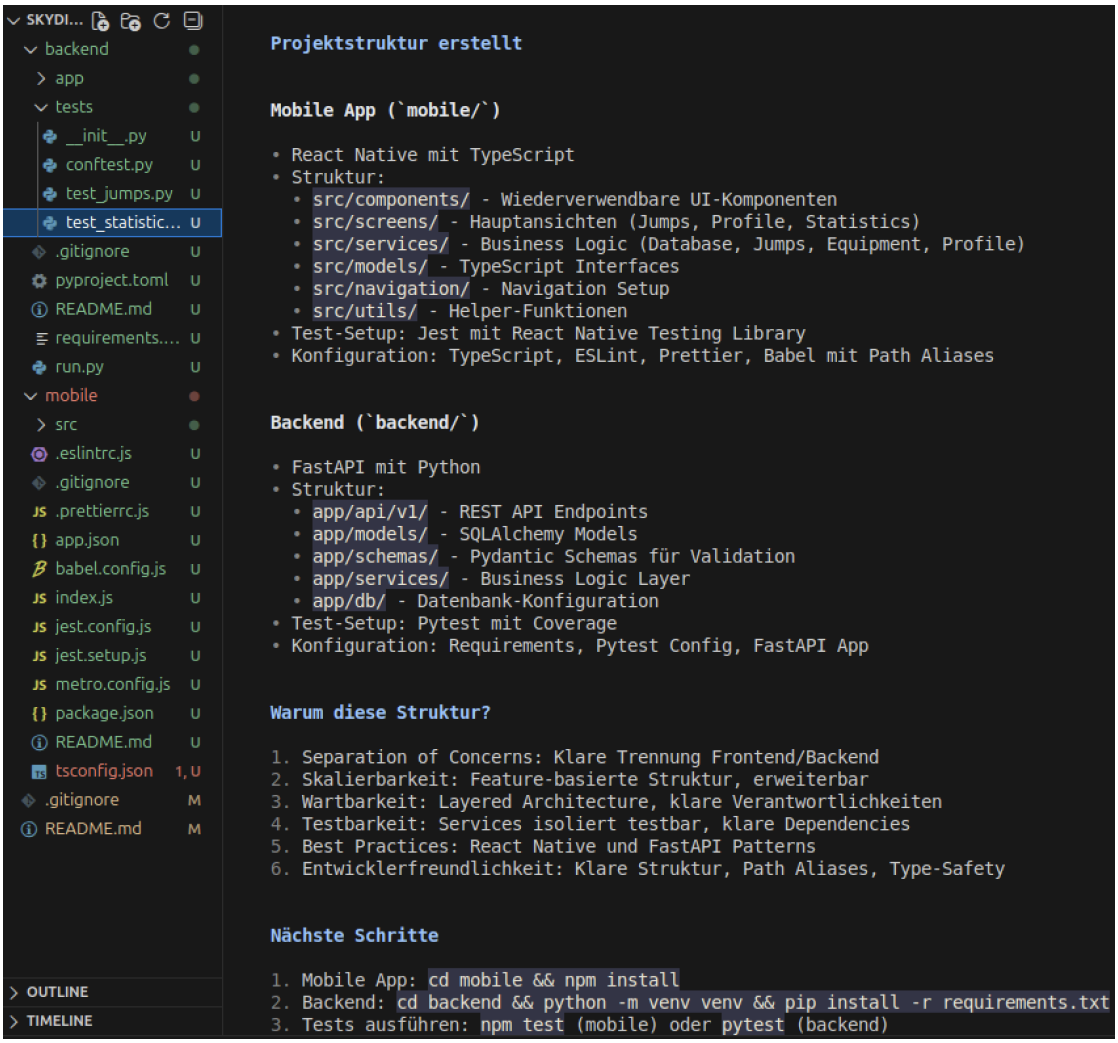


Abbildung 1: Erste Cursor-Agent-Aktion - Aufbau Projektstruktur

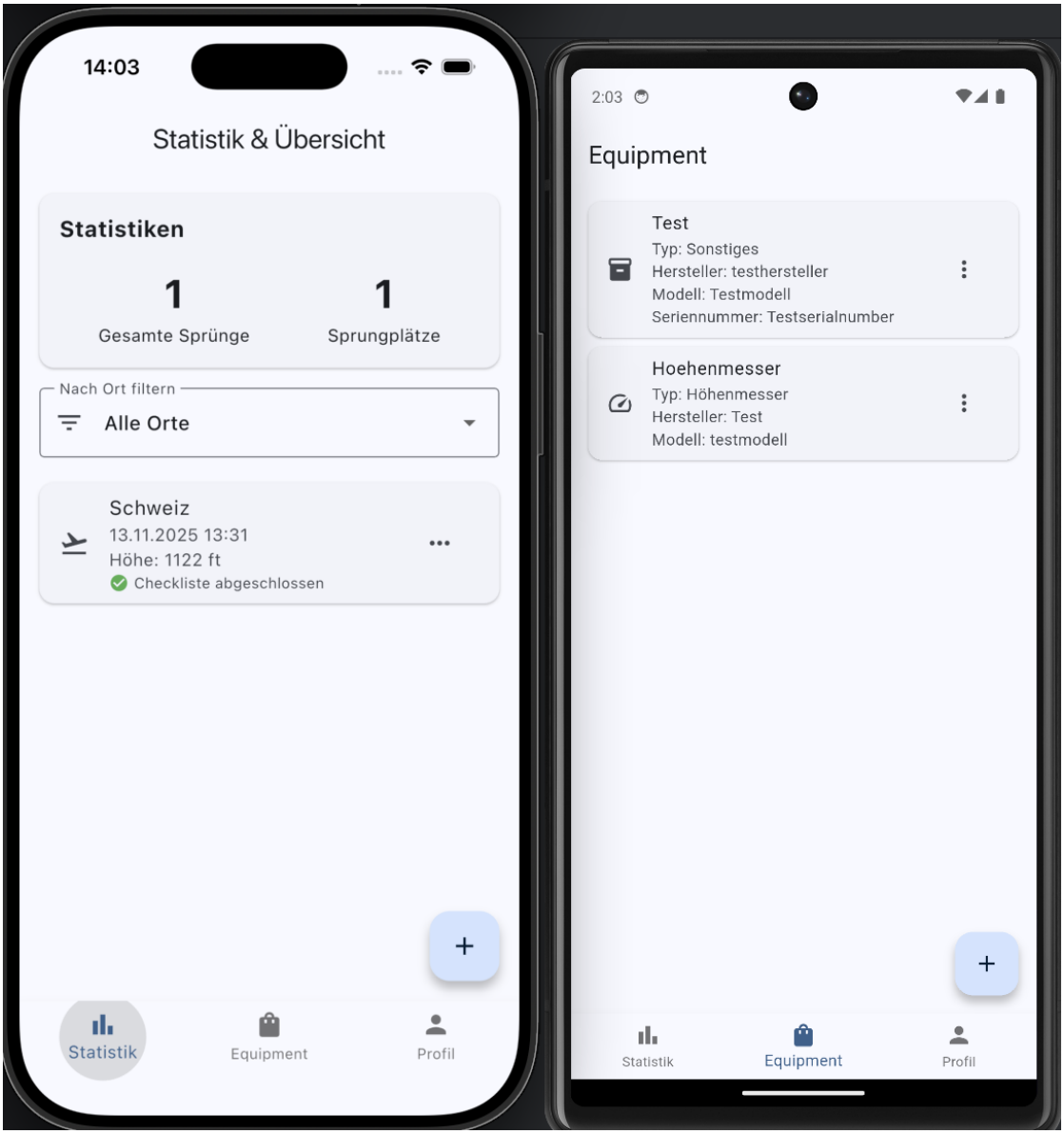


Abbildung 2: Erste Funktionen

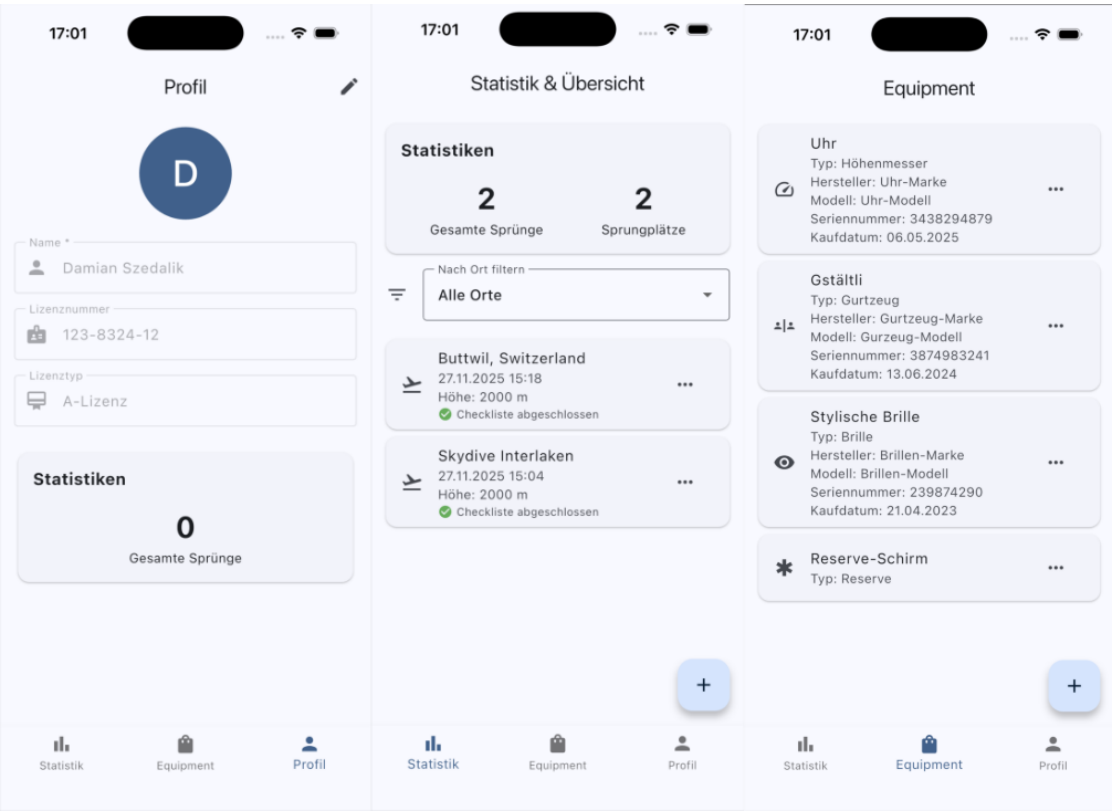
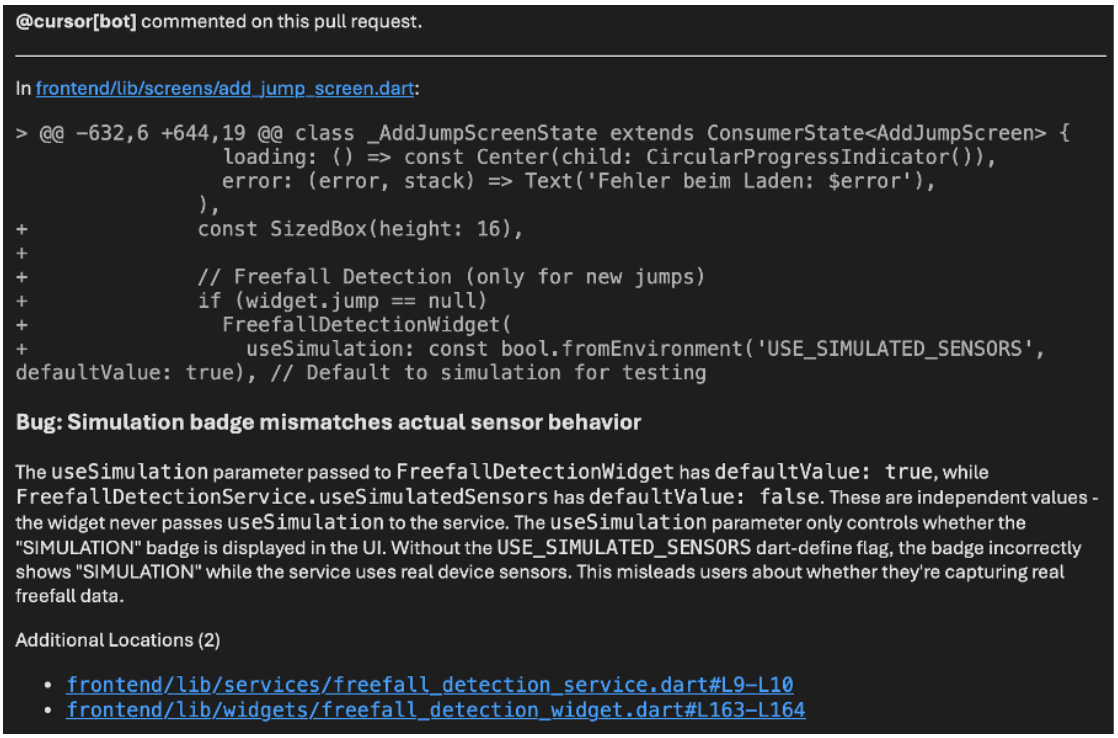


Abbildung 3: Erste Funktionen in Applikation vorhanden



@cursor[bot] commented on this pull request.

In frontend/lib/screens/add_jump_screen.dart:

```
> @@ -632,6 +644,19 @@ class _AddJumpScreenState extends ConsumerState<AddJumpScreen> {
                  loading: () => const Center(child: CircularProgressIndicator()),
                  error: (error, stack) => Text('Fehler beim Laden: $error'),
                ),
+               const SizedBox(height: 16),
+
+               // Freefall Detection (only for new jumps)
+               if (widget.jump == null)
+                 FreefallDetectionWidget(
+                   useSimulation: const bool.fromEnvironment('USE_SIMULATED_SENSORS',
defaultValue: true), // Default to simulation for testing
```

Bug: Simulation badge mismatches actual sensor behavior

The `useSimulation` parameter passed to `FreefallDetectionWidget` has `defaultValue: true`, while `FreefallDetectionService.useSimulatedSensors` has `defaultValue: false`. These are independent values - the widget never passes `useSimulation` to the service. The `useSimulation` parameter only controls whether the "SIMULATION" badge is displayed in the UI. Without the `USE_SIMULATED_SENSORS` dart-define flag, the badge incorrectly shows "SIMULATION" while the service uses real device sensors. This misleads users about whether they're capturing real freefall data.

Additional Locations (2)

- frontend/lib/services/freefall_detection_service.dart#L9-L10
- frontend/lib/widgets/freefall_detection_widget.dart#L163-L164

Abbildung 4: Kommentar zu einem Pull Request

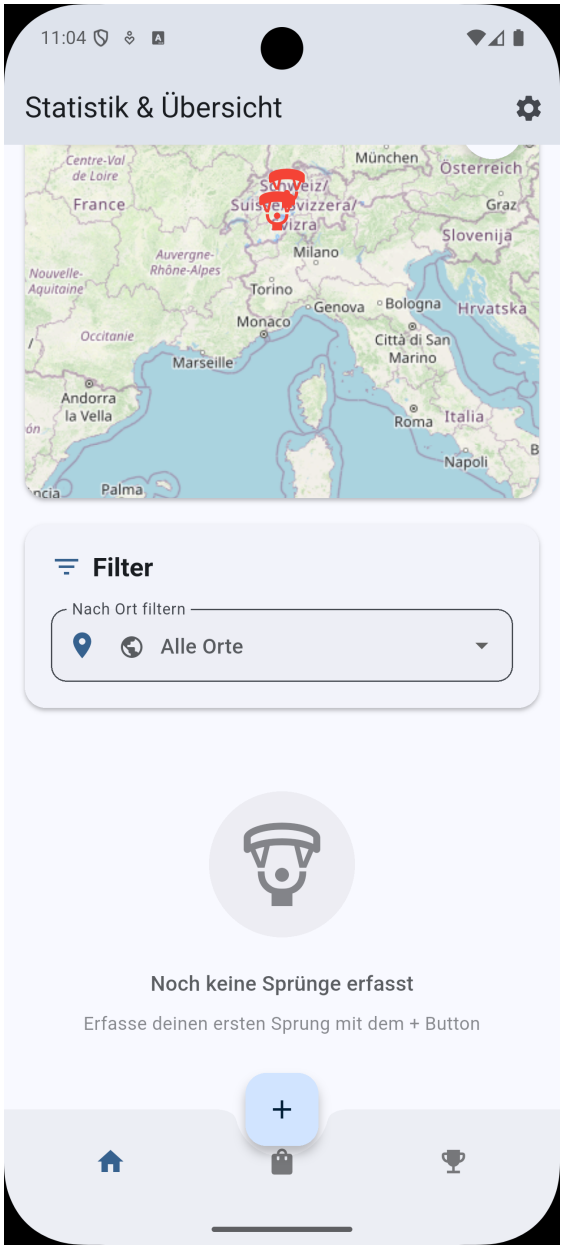


Abbildung 5: Filter-Problem

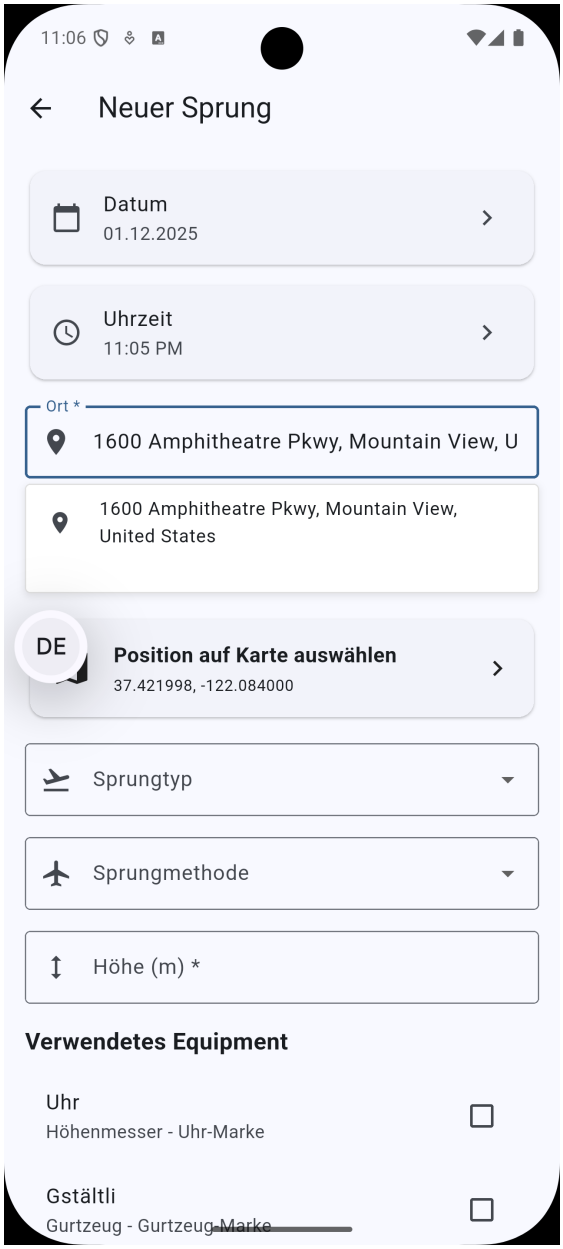


Abbildung 6: Location-Eingabe-Problem

Abbildung 7: Wetter-Implementation

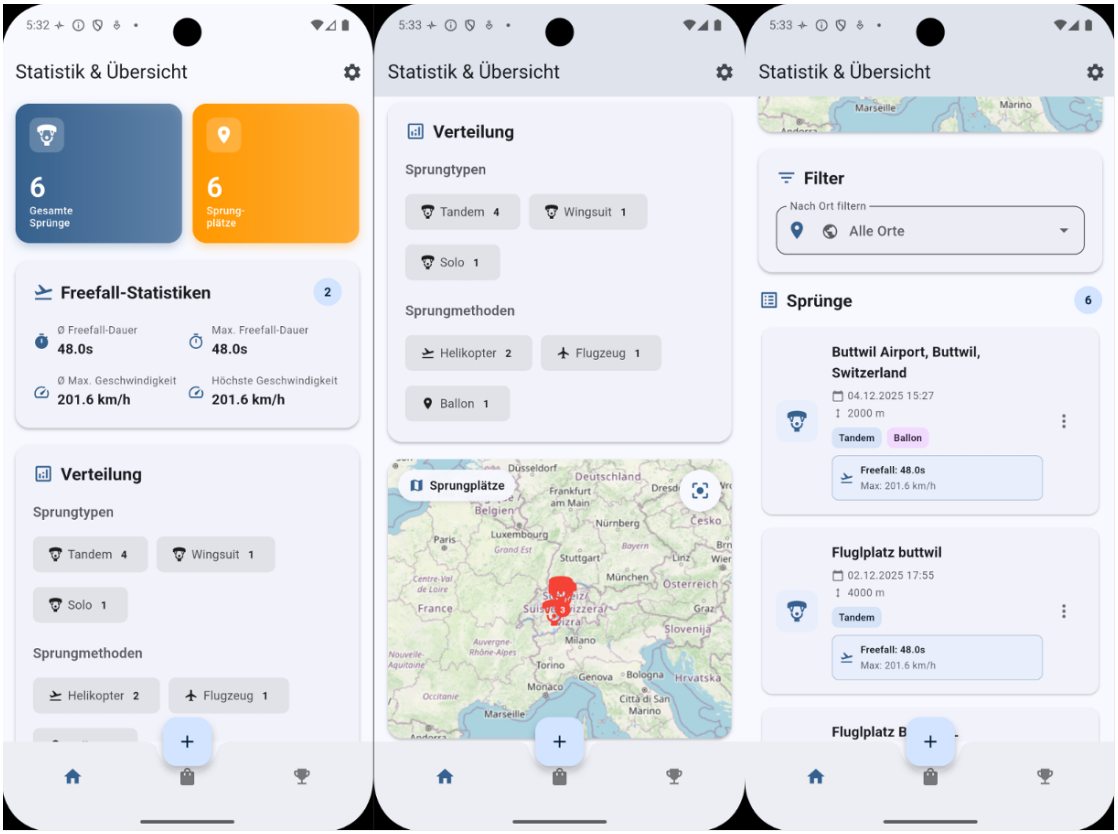


Abbildung 8: Statistik-Seite

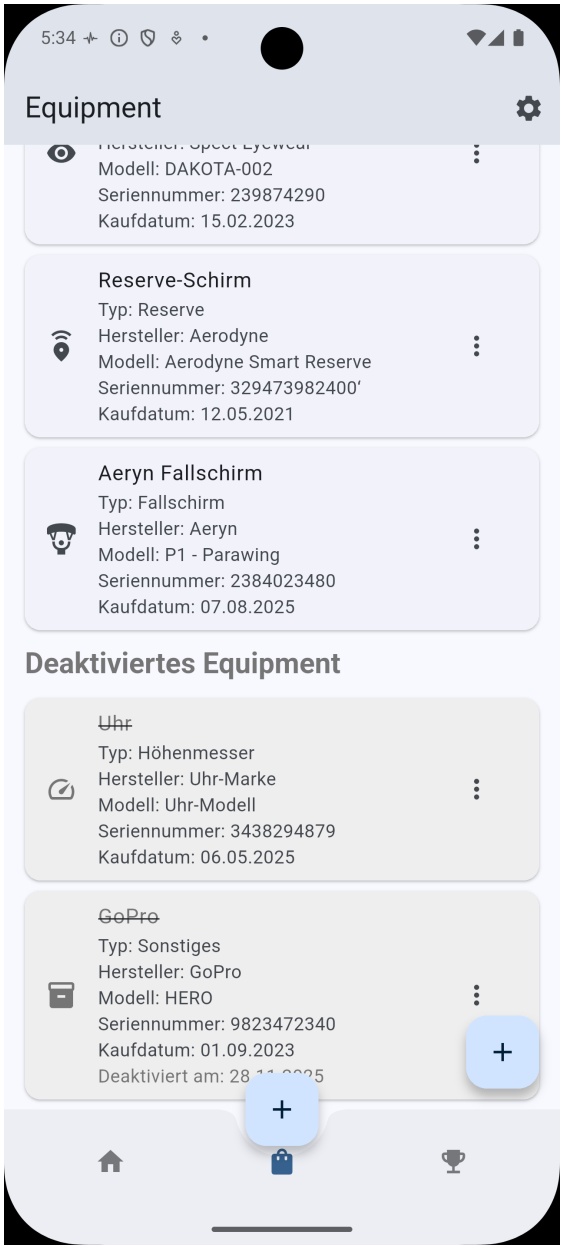


Abbildung 9: Equipment-Seite

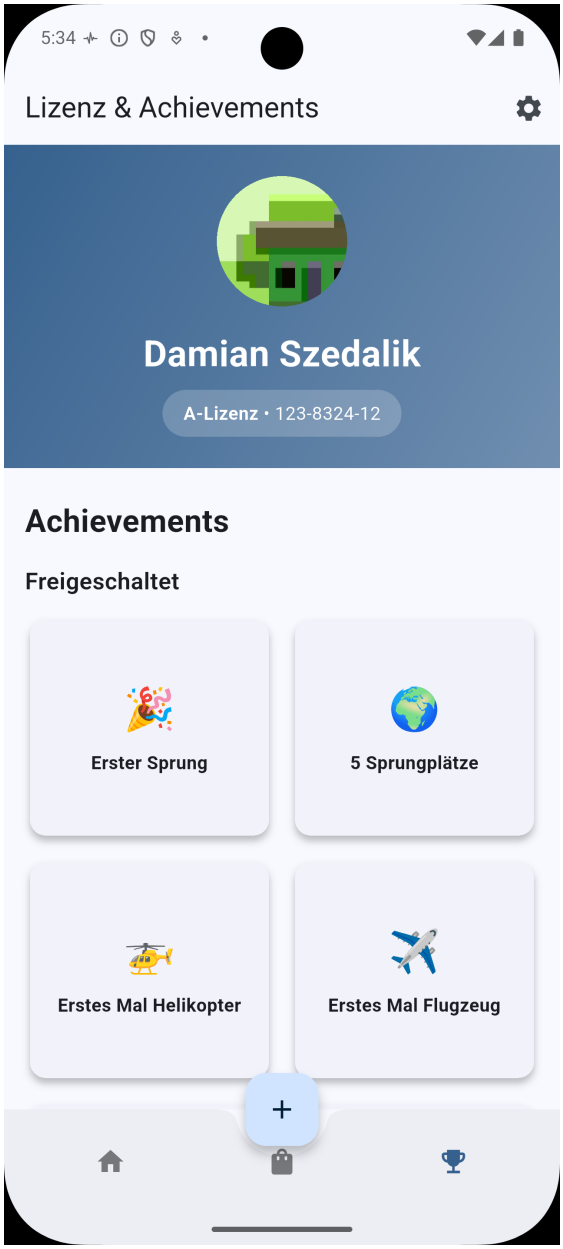


Abbildung 10: Lizenz- und Achievement-Seite

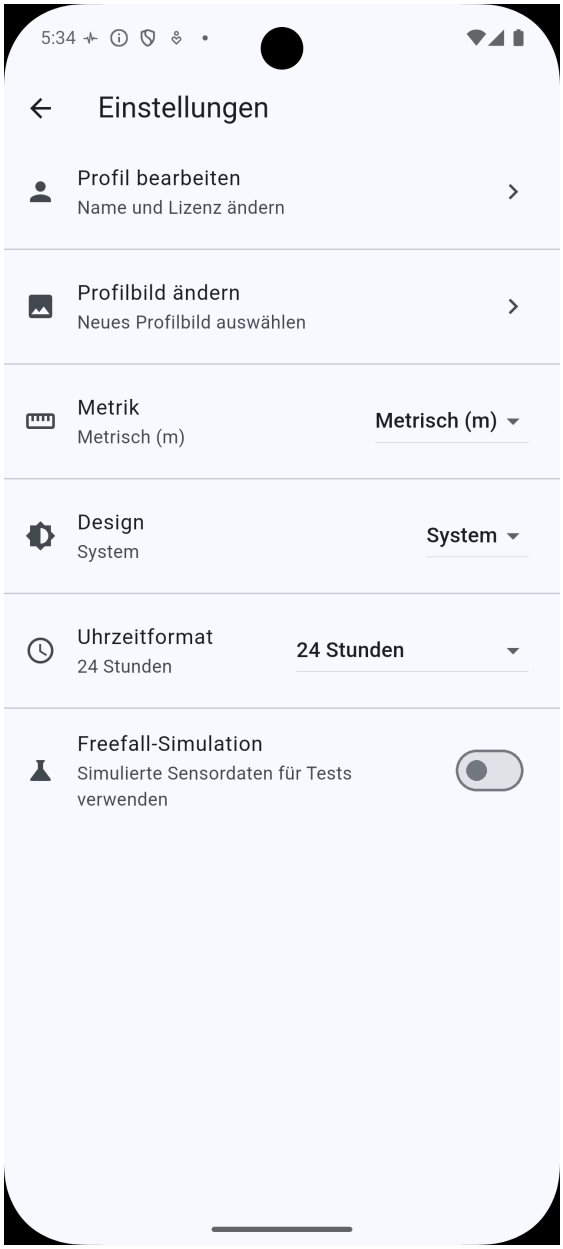


Abbildung 11: Einstellungs-Seite

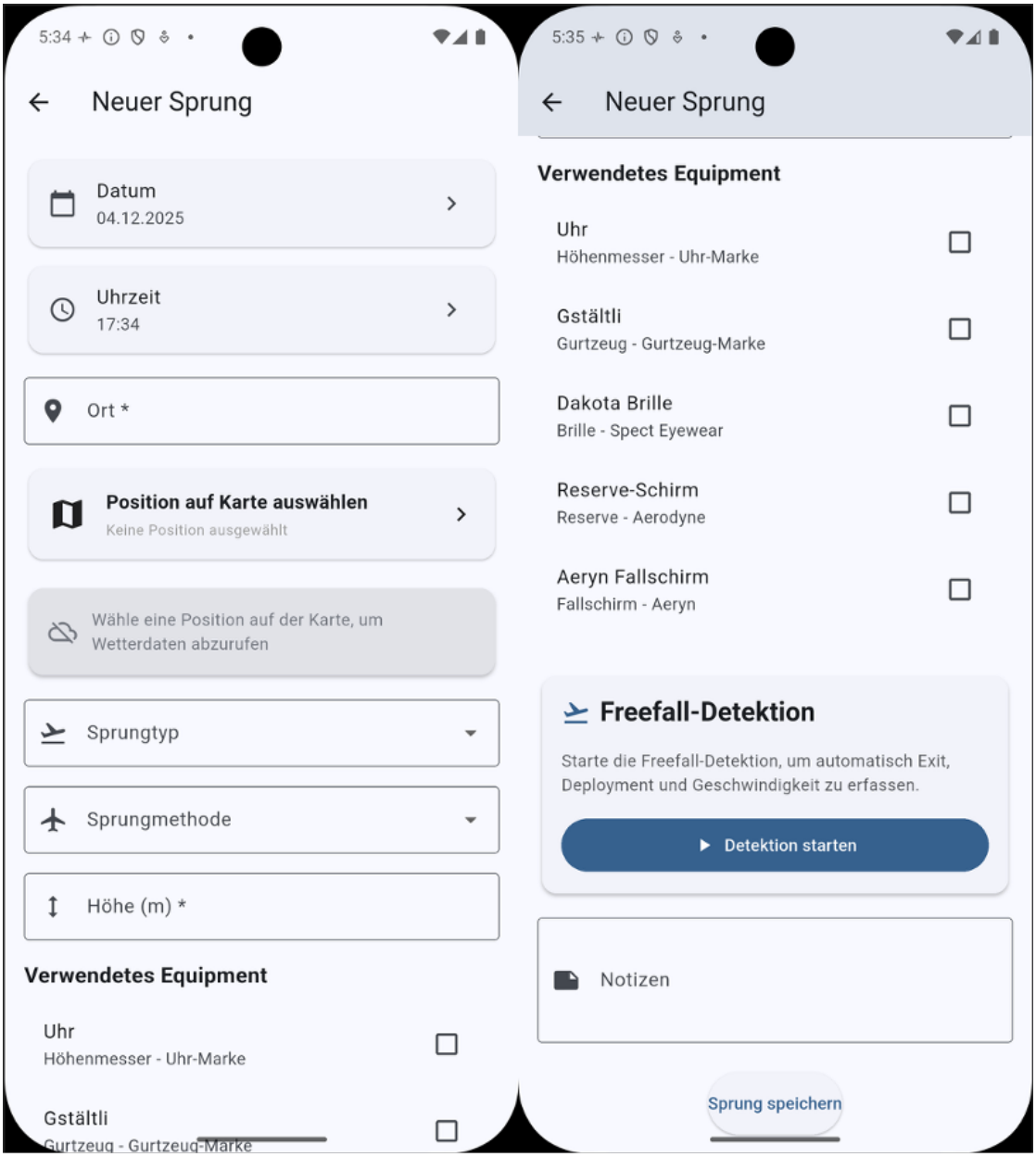


Abbildung 12: Sprungerfassungs-Seite